Engenharia de Software

Definição

- O que é
  - Aplicação de uma abordagem sistemática, disciplinada e quantificável para o desenvolvimento, operação e manutenção de software.
  - Estudo de abordagens para essas aplicações.
- Objetivo
  - Produzir software de alta qualidade.
  - Atender aos requisitos do usuário.
  - Dentro do prazo e orçamento.
  - De forma eficiente e eficaz.

Áreas de Conhecimento

- Requisitos de Software
  - Elicitação: Coleta de necessidades do usuário (entrevistas, workshops).
  - Análise: Entendimento e refinamento dos requisitos.
  - Especificação: Documentação clara e precisa dos requisitos (casos de uso, histórias de usuário).
  - Validação: Confirmação de que os requisitos estão corretos e completos.
- Projeto de Software
  - Arquitetura de Software: Estrutura geral do sistema (camadas, módulos).
  - Projeto Detalhado: Projeto de componentes individuais, interfaces, estruturas de dados.
  - Padrões de Projeto: Soluções reutilizáveis para problemas comuns (MVC, Observer).
- Construção de Software
  - Codificação: Implementação do projeto em uma linguagem de programação.
  - Teste de Unidade: Verificação de componentes individuais.
  - Refatoração: Melhoria da estrutura interna do código sem alterar o comportamento externo.
- Teste de Software
  - Teste de Integração: Verificação da interação entre componentes.
  - Teste de Sistema: Verificação do sistema completo em relação aos requisitos.
  - Teste de Aceitação: Verificação pelo cliente para garantir que o sistema atende às suas expectativas.
  - Tipos de Teste: Funcional, não funcional (desempenho, segurança).
- Manutenção de Software
  - Manutenção Corretiva: Correção de defeitos.
  - Manutenção Adaptativa: Adaptação do software a novos ambientes (hardware, SO).
  - Manutenção Evolutiva: Adição de novas funcionalidades ou melhorias.
  - Manutenção Preventiva: Melhoria da qualidade do código para prevenir problemas futuros.
- Gerenciamento de Configuração de Software (SCM)
  - Controle de Versão: Gerenciamento de alterações no código-fonte (Git, SVN).
  - Gerenciamento de Releases: Planejamento e execução de lançamentos de software.
  - Auditoria de Configuração: Verificação da integridade dos itens de configuração.
- Gerenciamento de Projetos de Software
  - Planejamento: Definição de escopo, cronograma, recursos e orçamento.
  - Monitoramento e Controle: Acompanhamento do progresso e gerenciamento de riscos.
  - Gerenciamento de Riscos: Identificação, análise e mitigação de riscos.
  - Gerenciamento de Equipe: Liderança e coordenação de desenvolvedores.
- Qualidade de Software
  - Garantia de Qualidade (QA): Processos para garantir a qualidade (revisões de código, inspeções).
  - Controle de Qualidade (QC): Atividades para verificar a qualidade (testes).
  - Métricas de Qualidade: Medidas para avaliar a qualidade do software (defeitos por linha de código).
- Processos de Engenharia de Software
  - Modelos de Processo: Cascata, Iterativo, Incremental.
  - Metodologias Ágeis: Scrum, Kanban, XP (eXtreme Programming).
  - DevOps: Integração de desenvolvimento e operações.

Paradigmas de Desenvolvimento

- Programação Estruturada
  - Foco em funções e procedimentos.
  - Uso de estruturas de controle (sequência, seleção, iteração).
- Programação Orientada a Objetos (POO)
  - Foco em objetos (encapsulamento, herança, polimorfismo).
  - Linguagens: Java, C#, Python, C++.
- Programação Funcional
  - Foco em funções puras e imutabilidade.
  - Linguagens: Haskell, Lisp, Erlang.
- Programação Baseada em Componentes
  - Reutilização de componentes de software pré-existentes.

Ferramentas

- IDEs (Ambientes de Desenvolvimento Integrado)
  - Visual Studio Code, IntelliJ IDEA, Eclipse.
- Sistemas de Controle de Versão
  - Git, Subversion (SVN).
- Ferramentas de Gerenciamento de Projetos
  - Jira, Trello, Asana.
- Ferramentas de Teste
  - Selenium, JUnit, Postman.
- Ferramentas de CI/CD (Integração Contínua/Entrega Contínua)
  - Jenkins, GitLab CI/CD, GitHub Actions.

Desafios

- Gerenciamento de Complexidade
  - Sistemas grandes e distribuídos.
- Mudança de Requisitos
  - Necessidade de adaptabilidade.
- Qualidade e Confiabilidade
  - Garantir software robusto e sem falhas.
- Segurança
  - Proteger contra ataques e vulnerabilidades.
- Produtividade
  - Otimizar o processo de desenvolvimento.

Ética e Responsabilidade Social

- Códigos de Conduta
  - IEEE/ACM Software Engineering Code of Ethics and Professional Practice.
- Implicações Sociais
  - Impacto do software na sociedade (privacidade, automação).
- Propriedade Intelectual
  - Direitos autorais, patentes, licenças de software.

Presented with xmind